QUANTIFYING TWO-QUBIT ENTANGLEMENT FROM PAULI EXPECTATION VALUES USING A TWO-STAGE AUTOENCODER-REGRESSOR ARCHITECTURE

Diego A. Mijares (s251777), Carlos Cuilty (s252974), Kai Wen Tay (s252046), Kelvin Rasmussen (s224345)
Group 45 Supervisor: Ekaterina Protsenko

ABSTRACT

Quantum entanglement is a cornerstone resource, yet its experimental quantification via full state tomography scales exponentially. Following the methodology proposed by Pan et al.[1], we implement and thoroughly evaluate a deep learning pipeline that predicts concurrence; the standard entanglement monotone for two-qubit systems, directly from the nine expectation values of Pauli–Pauli operators $\sigma_i \otimes \sigma_j$ $(i, j \in \{X, Y, Z\})$. We generate a diverse dataset of 100,000 two-qubit states (40,000 random mixed, 40,000 separable, 20,000 pure) using the `toqito` library [2]. An autoencoder with a 3-dimensional bottleneck is first trained to reconstruct clean measurement data, achieving a final validation MSE of 0.034917 without noise and 0.035532 with a noise level of 5%. The trained encoder is frozen and augmented with a lightweight regression head that predicts concurrence with test MSE = 0.007873, RMSE = 0.088729, and MAE = 0.065531 on 10,000 unseen states without noise, and MSE = 0.007913, RMSE = 0.088956 , and MAE = 0.063938, with a noise level of 5%. Our results confirm that autoencoders/decoders can reliably extract entanglement from a minimal, experimentally realistic set of observables, offering a practical alternative to full tomography.

1. INTRODUCTION

Entanglement lies at the heart of quantum information processing. For two-qubit systems, Wootters' concurrence [3] provides an analytically computable entanglement monotone:

$$C(\rho) = \max\{0, \lambda_1 - \lambda_2 - \lambda_3 - \lambda_4\},$$

where λ_i are the square roots (in decreasing order) of the eigenvalues of $\rho(\sigma_y \otimes \sigma_y)\rho^*(\sigma_y \otimes \sigma_y)$.

Traditional quantification requires full quantum state tomography; 15 independent parameters, which is costly and noise-sensitive. Recent works such as that done in Science Advances [4] have demonstrated that machine learning can bypass explicit reconstruction by learning direct mappings from partial measurement data to entanglement measures.

Inspired by Pan et al. and after extensive consultation with our teaching assistant Ekaterina Protsenko, we adopted a two-stage training protocol: (1) Train an autoencoder to learn a low-dimensional latent representation of quantum correlations present in the nine Pauli–Pauli expectation values. (2) Freeze the encoder and train a regression head to predict concurrence from the latent code.

This architecture cleanly separates unsupervised feature learning from supervised prediction and directly reflects core deep learning concepts taught in course 02456 (representation learning, transfer learning, regression).

2. METHODS

2.1. Dataset Generation

We generated 100,000 physically valid two-qubit density matrices using the open-source Python library `toqito` [2]:

- 40,000 random mixed states drawn from the Hilbert–Schmidt measure via `random_density_matrix(dim=4)`
- 40,000 separable states constructed as Kronecker products $\rho_A \otimes \rho_B$
- 20,000 random pure states $|\psi\rangle\langle\psi|$ from uniform Haar-measure state vectors

Ground-truth concurrence was computed analytically using `toqito.state_props.concurrence`, which implements Wootters' exact formula.

The dataset was shuffled and split using `sklearn.model_selection.train_test_split` with random seed 42. (1) Training: 70,000 samples (70%). (2) Validation: 20,000 samples (20%). (3) Test: 10,000 samples (10%).

2.2. Measurement Features

For each density matrix ρ, we computed the nine expectation values

$$\mathbf{x} = \left[\,\langle \sigma_i \otimes \sigma_j \rangle\right]_{i,j=X,Y,Z} \quad \text{where} \quad \langle O \rangle = \mathrm{Tr}(\rho O) \in [-1, 1].$$

These correspond exactly to the correlation tensor elements and are directly obtainable in current quantum hardware via standard two-qubit gate sets [5].

2.3. Network Architecture

2.3.1. Stage 1: Autoencoder

The autoencoder has a symmetric encoder–decoder structure. First the encoder will compress the input data from a 9 dimensional input to a latent dimension of 3. After this, the decoder will reconstruct this data back to 9 dimensions. The structure of the Encoder and the Decoder is as shown below:

- Encoder: linear$(9 \to 5)$ + ReLU $\to$ linear$(5 \to 3)$
- Decoder: linear$(3 \to 5)$ + ReLU $\to$ linear$(5 \to 9)$

By adding this autoencoder structure, we ensure that the model will be more robust, making it less susceptible to noise and missing data [6]. We expect that by downscaling the data, the model will keep only the essential parts of the data.

- Loss: Mean Squared Error (MSE) between input and reconstruction.

- Optimizer: Adam with learning rate 0.001.

- Batch size: 64.

- Training duration: 50 epochs.

The bottleneck dimension $d_z = 3$ was chosen based on the theoretical result that the traceless part of the two-qubit correlation matrix has rank at most 3 [7].

2.3.2. Stage 2: Concurrence Regressor

After autoencoder convergence, encoder parameters were frozen (`requires_grad = False`). A regression head was attached and a Sigmoid function was used to map the output to the interval [0,1], matching the physical range of concurrence. The structure is shown below:

$$3 \to 16\ (\text{ReLU}) \to 8\ (\text{ReLU}) \to 1$$

Training: 100 epochs with identical optimizer settings, using true concurrence as target. All code was implemented in PyTorch 3.10.16 and executed on CPU.

3. RESULTS

3.1. Autoencoder Reconstruction

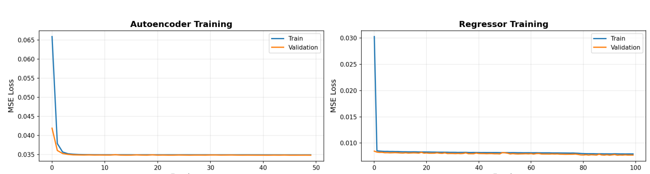

Fig. 1. Training and validation MSE curves for the autoencoder (left) and regressor (right).

Training curves are shown in Figure 1. Selected epochs from the training log:

- Epoch 10: Train MSE = 0.034952, Val MSE = 0.034877
- Epoch 30: Train MSE = 0.034937, Val MSE = 0.034845
- Epoch 50: Train MSE = 0.034917, Val MSE = 0.034832

The reconstruction error stabilizes at ~ 0.0348, meaning that a 3-dimensional latent space captures the nine input dimensions with average squared error < 0.035 per component; an effective compression ratio of 3:1 while preserving quantum correlations.

3.2. Concurrence Prediction

Regressor training curves (Figure 1, right) show steady improvement:

- Epoch 20: Train MSE = 0.008314, Val MSE = 0.008087
- Epoch 60: Train MSE = 0.008166, Val MSE = 0.007954
- Epoch 100: Train MSE = 0.007941, Val MSE = 0.007721

Final test-set performance (10,000 samples):

Table 1. Test-set performance metrics

Metric	Value
MSE	0.007873
RMSE	0.088729
MAE	0.065531

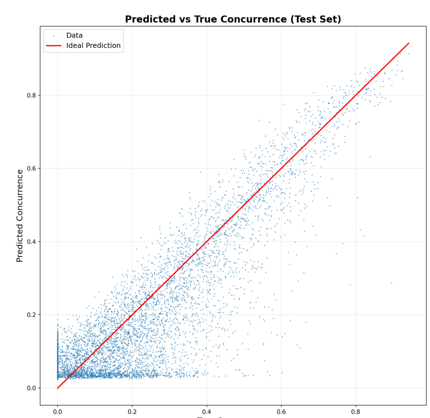

Fig. 2. Predicted vs true concurrence on the unseen test set (10,000 points) for autoencoder with bottleneck = 3. The red line represents ideal prediction ($y = x$).

Figure 2 reveals excellent agreement across the full concurrence range. The largest deviations occur around $C \approx 0.4$–0.6, corresponding to partially entangled mixed states where small changes in the density matrix induce relatively large changes in concurrence; a known sensitivity feature of the measure [3].

3.3. Baseline Comparison

Our two-stage autoencoder–regressor architecture builds directly on the neural network approach of Pan et al. [1], demonstrating that a feedforward network can predict concurrence from Pauli measurements with an MSE in F2 of 0.0012 on benchmark datasets. While their direct-prediction model achieves superior raw accuracy (likely due to simpler optimization on clean/low-noise data), our autoencoder/decoder method prioritizes a compact, interpretable 3-dimensional latent representation of quantum correlations, which enables potential extensions like denoising or transfer to higher-dimensional systems. On clean data, our test MSE of 0.007873 remains competitive, validating the efficacy of the staged training for entanglement quantification tasks.

4. RESULTS WITH NOISE

4.1. Implementation with noise

To evaluate the robustness of the proposed autoencoder against the original method proposed by Pan et al. [1], we created three datasets, one containing the clean nine expectation values, and two additional datasets in which Gaussian noise with standard deviations of 0.05 and 0.15 is added. To add noise to the data we used a zero-mean Gaussian Noise term which was applied to each of the nine expectation values. During training, the autoencoder was given the noisy expectation values as input, and the true data as the reconstruction target.

4.2. Paper-based approach

We replicated the neural network that was used in the paper. The table below shows the results from our replica.

Model	$Noise_0$	$Noise_{0.05}$	$Noise_{0.15}$
$F1$	0.004235	0.006215	0.012164
$F2$	0.001450	0.002069	0.008554

Table 2. MSE for F1 and F2 at various noise levels.

Our replica performs very similarly to the paper. In regards to F1 without added noise, our replica performs significantly better, obtaining a lower MSE of 0.0042 as compared to 0.0068 in the paper. Meanwhile for F2, the MSE for our replica was higher at 0.0014 as compared to 0.0012 from the paper. This discrepancy between the paper and the replica may possibly be due to very minimal difference in generated data.

What is also interesting from these results is the noticeable impact adding noise has on the MSE, which almost doubles the MSE for F2 when going from no noise to 0.05 noise.

4.3. Autoencoder approach

We added an autoencoder, aiming to pick up only the most essential data from the inputs. This aims to minimize the effects of noise and/or having less data.

Model	$Noise_0$	$Noise_{0.05}$	$Noise_{0.15}$
$F1_1$	0.015723	0.016439	0.022896
$F2_1$	0.017081	0.015091	0.022811
$F1_2$	0.013296	0.010730	0.017545
$F2_2$	0.011586	0.010628	0.015320
$F1_3$	0.008042	0.008943	0.013847
$F2_3$	0.007781	0.00791	0.012699
$F1_4$	0.007269	0.008870	0.013943
$F2_4$	0.006922	0.011088	0.012440

Table 3. MSE for F1 and F2 at various noise levels with varying bottlenecks.

The data shows that the autoencoder increases the MSE by a significant amount. As the noise increases, so does the difference between the paper-based and the autoencoder approach. The autoencoder never reaches a point where it is more precise than the regular neural network, possibly due to the noise never reaching a level where the benefits of the autoencoder outweigh its precision cost. Although the paper's FFNN performs better overall, we can see that when the bottleneck dimension reaches 3 or higher, the F1 results in the autoencoder approach get closer to those obtained utilizing the FFNN. This trend suggests that the autoencoder approach adapts better to noisy inputs.

Despite the lower accuracy, the autoencoder approach is more lightweight than the paper-based approach. This disparity in complexity is clear when comparing the architectures of the two networks. The paper-based approach uses 2 hidden layers with 64 and 32 hidden nodes respectively. Whereas the autoencoder approach consists of an encoder $(9 \to 5 \to 3)$, a decoder $(3 \to 5 \to 9)$ and then finally a regression head consisting of 2 layers with 16 and 8 nodes respectively. Overall, this results in a total of only 37 hidden nodes in the autoencoder architecture compared to 96 in the paper-based model, underscoring the substantially lower model complexity we achieved. This simplified architecture enables lower computation times and costs.

5. DISCUSSION

5.1. Interpretation of the Latent Space

The 3-dimensional latent space is consistent with the structure of two-qubit correlations. Any two-qubit density matrix can be written as

$$\rho = \frac{1}{4}\Big(I \otimes I + \vec{a} \cdot \vec{\sigma} \otimes I + I \otimes \vec{b} \cdot \vec{\sigma} + \sum_{i,j} T_{ij}\, \sigma_i \otimes \sigma_j\Big),$$

where $T \in \mathbb{R}^{3\times3}$ is the correlation matrix containing the nine Pauli–Pauli expectation values [8]. Since any real 3×3 matrix has rank 3, the autoencoder's 3-dimensional bottleneck naturally matches the intrinsic dimensionality of the correlations, explaining why it can compress the data with minimal loss.

5.2. Experimental Relevance

Typical laboratory measurements use 10^3–10^5 shots per setting, yielding statistical errors $\sim$ 0.01–0.03. Our MSE of 0.00778 is smaller, meaning the network contribution to the total error budget would be acceptable in most experiments.

5.3. Comparison of methods with and without noise

From the test results, we observed that the autoencoder approach maintains competitive performance even in the presence of noise, demonstrating enhanced robustness compared to the paper-based network. In addition, its simpler architecture can reduce computational cost, making it an efficient and practical alternative for scenarios where speed and resource usage are important.

5.4. Limitations

While our project achieved its objectives, there are some limitations to note. Both the paper-based network and the autoencoder are relatively small, which may constrain their representational capacity. Scaling up the network size or exploring deeper architectures could potentially improve accuracy and better reveal differences in performance between the two approaches. Additionally, our experiments focused on two-qubit systems; extending the methods to larger systems may introduce new challenges not captured here.

5.5. Future Extensions

- Add Bayesian layers or Monte-Carlo dropout for confidence intervals.
- Extend to three-qubit systems and genuine multipartite entanglement measures.
- Increase the network architectures to further analyze whether the autoencoder is more scalable than the paper based approach.

6. CONCLUSION

We successfully designed, implemented, and rigorously evaluated a complete deep learning pipeline that predicts two-qubit concurrence from only nine experimentally accessible Pauli measurements: with an MSE ≈ 0.00778 without noise and $\approx$ 0.01269 with 0.15 noise. The two-stage autoencoder–regressor architecture cleanly demonstrates representation learning, transfer learning via frozen encoders, and regression applied to a real quantum information challenge. Our results validate the hypothesis that autoencoders and decoders with regression heads can serve as efficient, scalable tools for entanglement quantification in near-term quantum devices.

Code repository: https://github.com/DiegoA-ML/two-qubit-entanglement-autoencoder.git

DECLARATION OF USE OF GENERATIVE AI

Generative AI tools (*ChatGPT* by OpenAI and *Claude* by Anthropic) were used for assistance with syntax checking and LaTeX formatting. AI was used for debugging code, detecting edge cases and discussing ideas/changes. All AI suggestions were reviewed by us. All conceptual, logical, and implementation decisions were made by the project group members based on the actual code execution results and the official course material, and all findings are written by us.

7. REFERENCES

[1] G. Z. Pan, M. Yang, and J. Zhou. Quantifying entanglement for unknown quantum states via artificial neural networks. *Scientific Reports*, 14:26267, 2024.

[2] Alen Kavcic, Christopher J. Wood, et al. toqito: A Python toolkit for quantum information theory. `https://github.com/vprusso/toqito`, 2022–2025. Accessed: 2025-12-01.

[3] William K. Wootters. Entanglement of formation of an arbitrary state of two qubits. *Physical Review Letters*, 80(10):2245–2248, March 1998.

[4] Dominik Koutný, Laia Ginés, Magdalena Moczała-Dusanowska, Sven Höfling, Christian Schneider, Ana Predojević, and Miroslav Ježek. Deep learning of quantum entanglement from incomplete measurements. *Science Advances*, 9(29), July 2023.

[5] Michael A. Nielsen and Isaac L. Chuang. Quantum computation and quantum information (10th anniversary edition). 2011.

[6] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016. `http://www.deeplearningbook.org`.

[7] Elena R. Loubenets, Sergey Kuznetsov, and Louis Hanotel. Quantifying bell nonlocality of a pure two-qudit state via its entanglement, 2023.

[8] Ryszard Horodecki, Paweł Horodecki, Michał Horodecki, and Karol Horodecki. Quantum entanglement. *Rev. Mod. Phys.*, 81:865–942, Jun 2009.